



Universidad Autónoma de Baja California

Programación Orientada a Objetos

Agregación, relación “has a”

Profesor: MC. Itzel Barriba Cázares

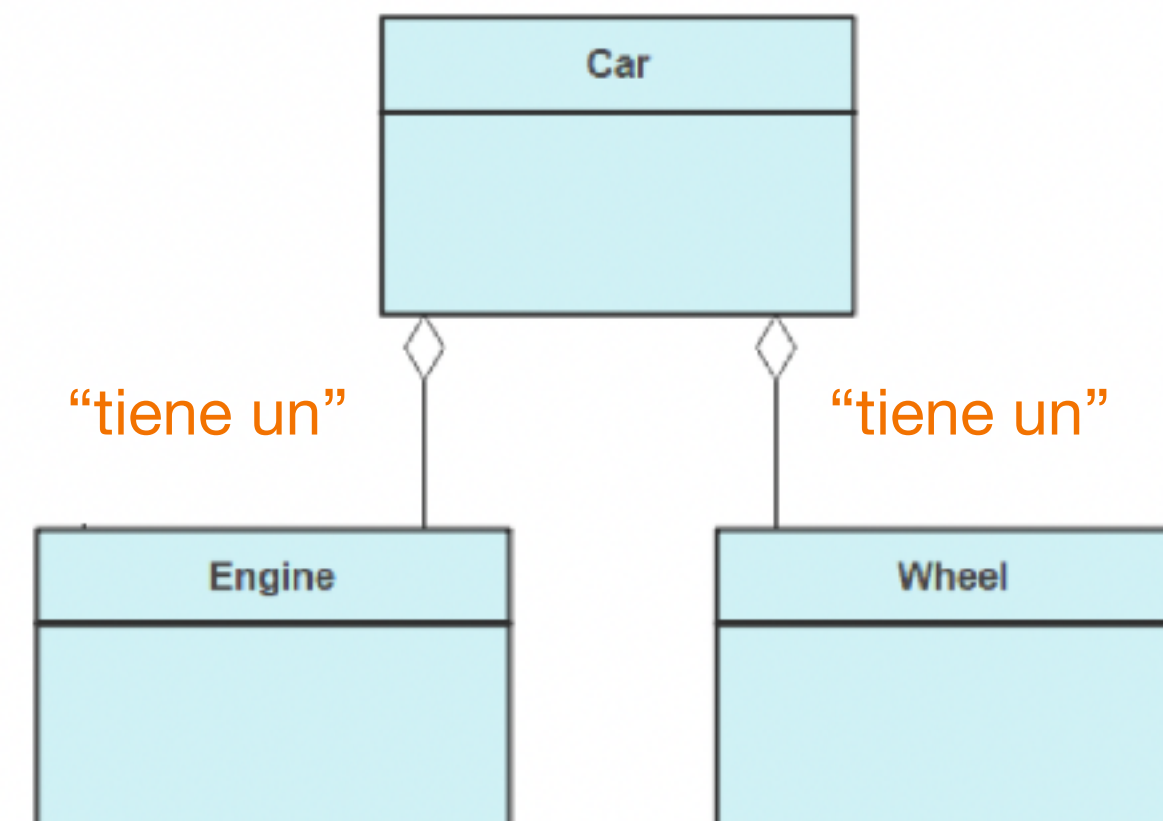
Agregación

- ▶ En java, la **agregación** es una relación en la que una clase contiene referencias a objetos de otra clase como parte de su estructura.
- ▶ Representa una **relación “tiene un”** y refleja cómo los objetos complejos pueden componerse de otros más simples, como una casa que tiene puertas, ventanas, paredes.



Agregación

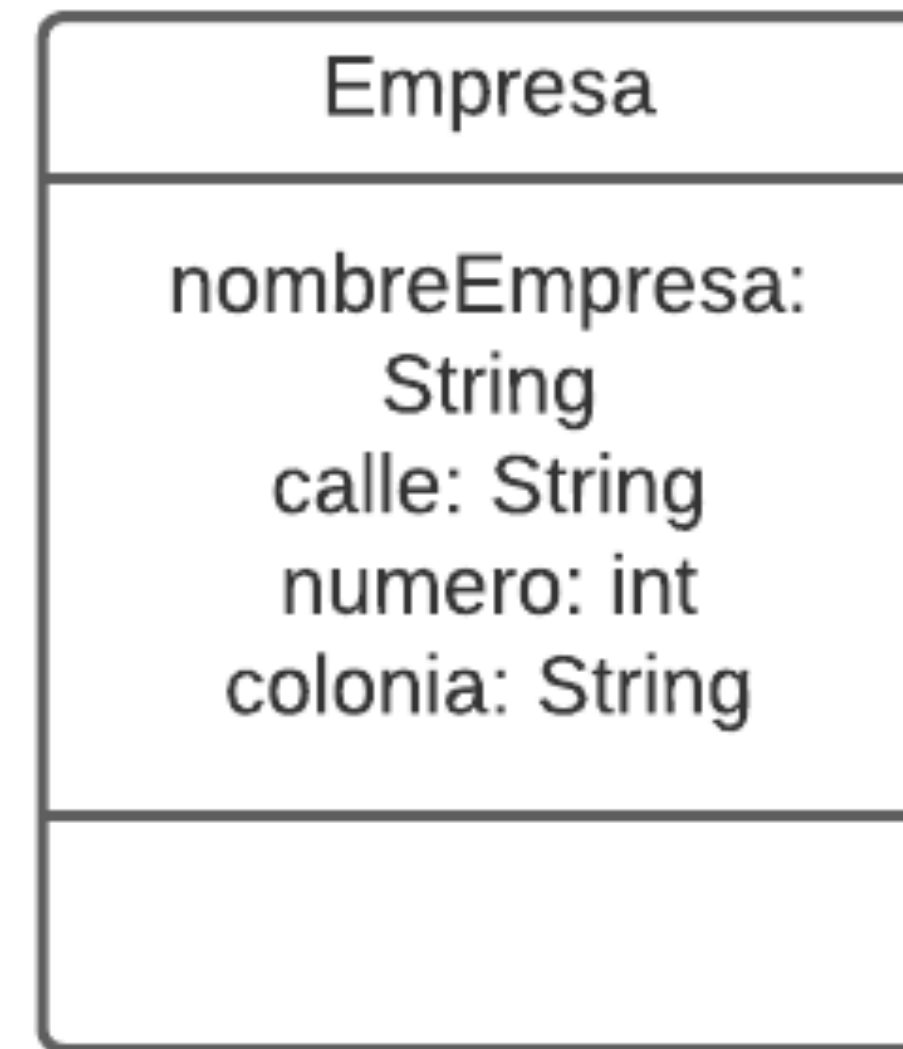
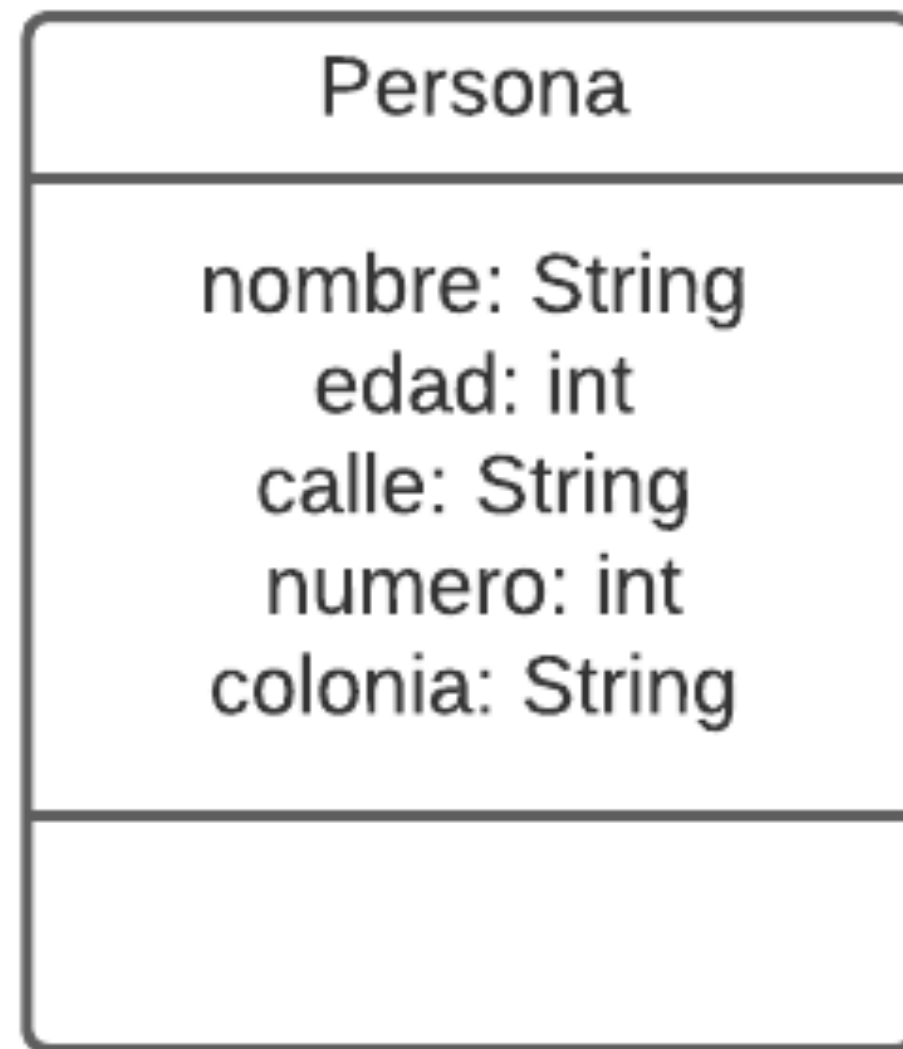
- En UML, la **agregación** se representa con una línea con un diamante vacío en el extremo de la clase contenedora. Indica una relacional “**todo/parte**”, en la que **el todo agrupa a las partes**, pero las partes pueden **existir independientemente**.



- En el caso del carro, elementos como el **motor** y las **llantas** son necesarios para su funcionamiento, pero **no dependen exclusivamente de él**, ya que pueden usarse en otros vehículos o máquinas.

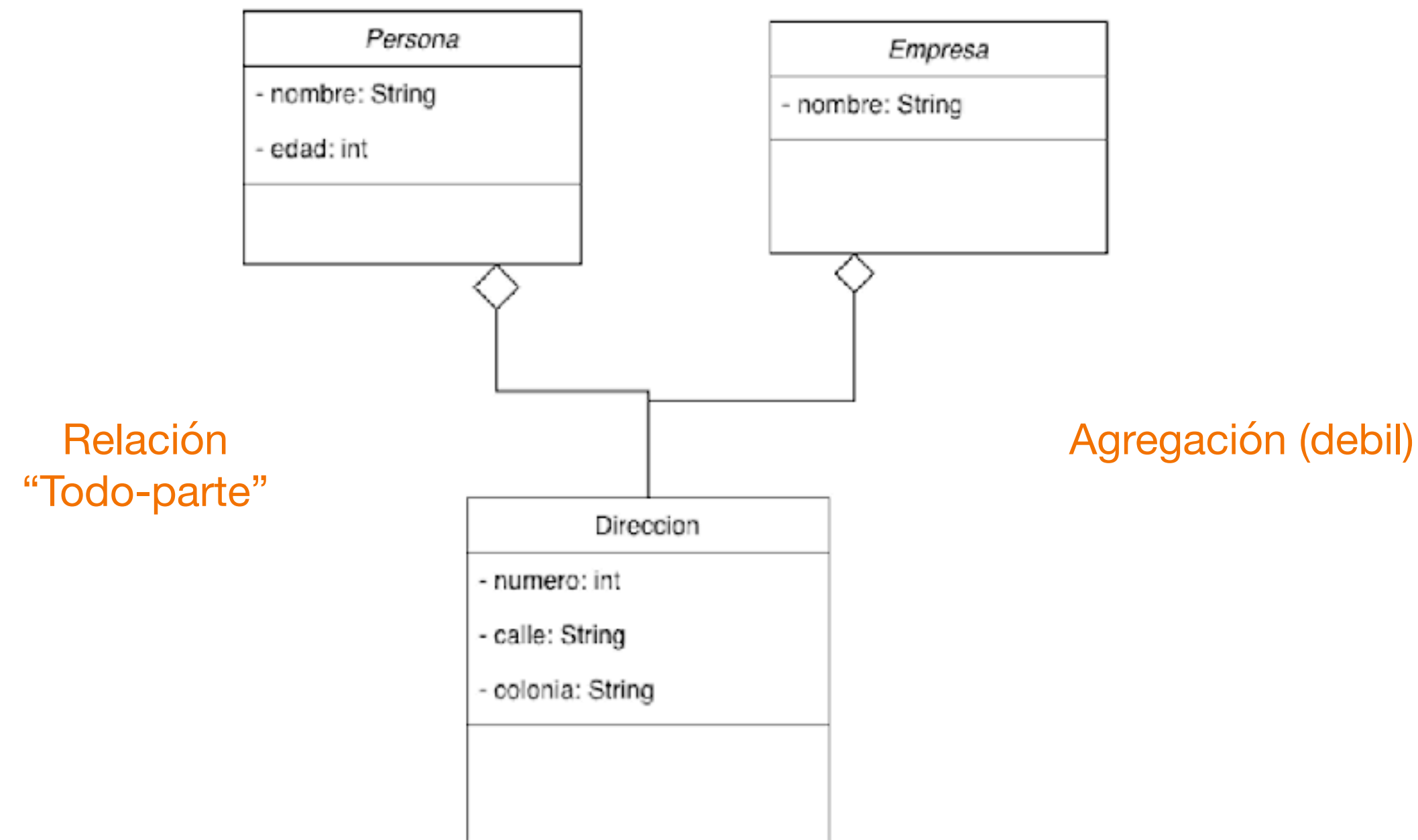
Agregación: Ejemplo(1)

- Por ejemplo, tenemos la clase Persona, Empresa



Tanto la Persona como la Empresa contienen
La misma información de dirección como atributos

Agregación: Ejemplo (1)

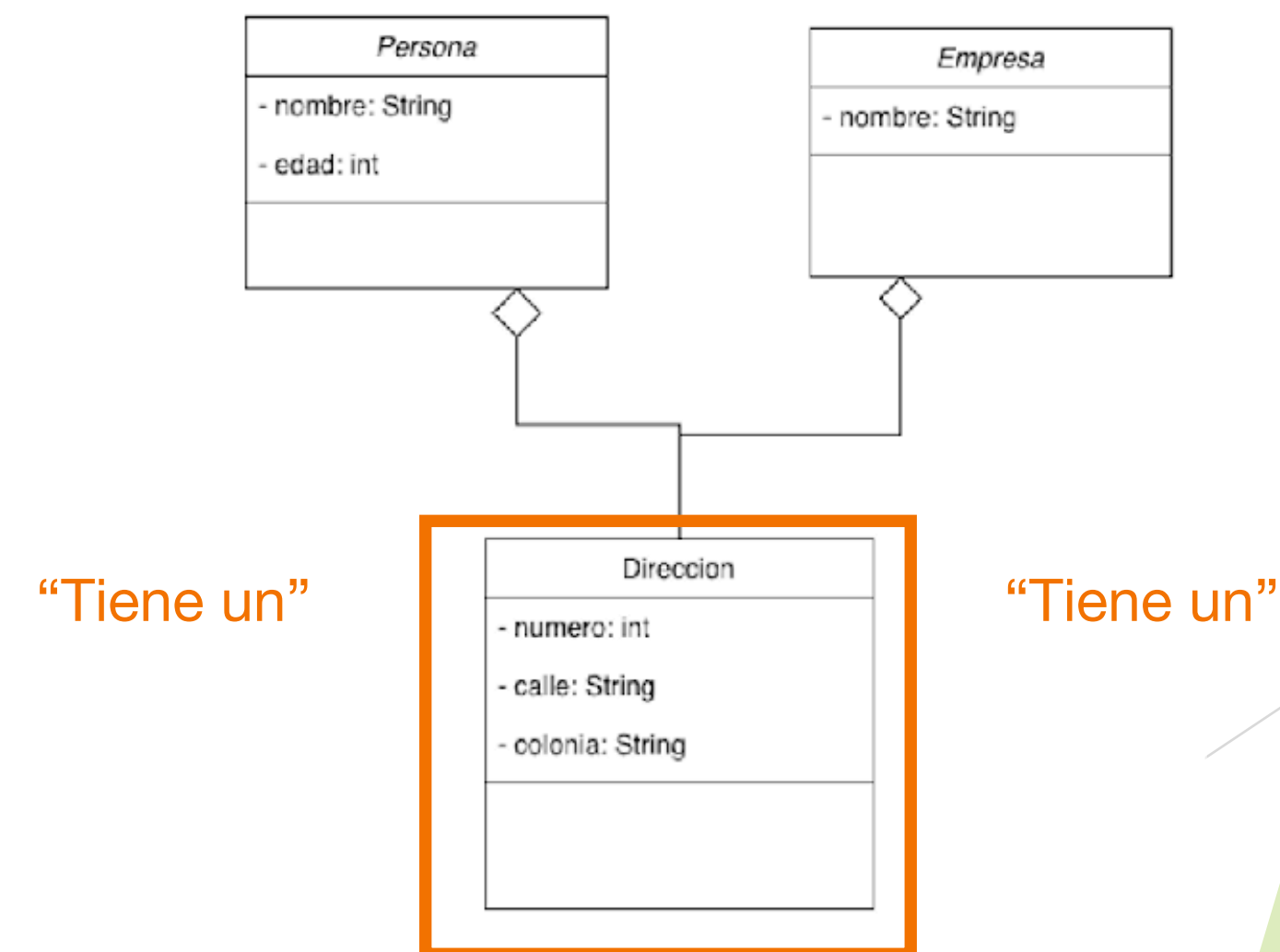


- ¿Puede existir Dirección sin Persona o Empresa?
- Si la respuesta es Si, entonces es agregación. Donde el objeto “tiene un” pero no lo controla.

Clase Dirección

```
public class Direccion {  
    private String calle;  
    private int numero;  
    private String colonia;  
  
    public Direccion(String calle, int numero, String colonia) {  
        this.calle = calle;  
        this.numero = numero;  
        this.colonia = colonia;  
    }  
    public String getCalle() {  
        return calle;  
    }  
  
    public void setCalle(String calle) {  
        this.calle = calle;  
    }  
    public int getNumero() {  
        return numero;  
    }  
  
    public void setNumero(int numero) {  
        this.numero = numero;  
    }  
    public String getColonia() {  
        return colonia;  
    }  
  
    public void setColonia(String colonia) {  
        this.colonia = colonia;  
    }  
}
```

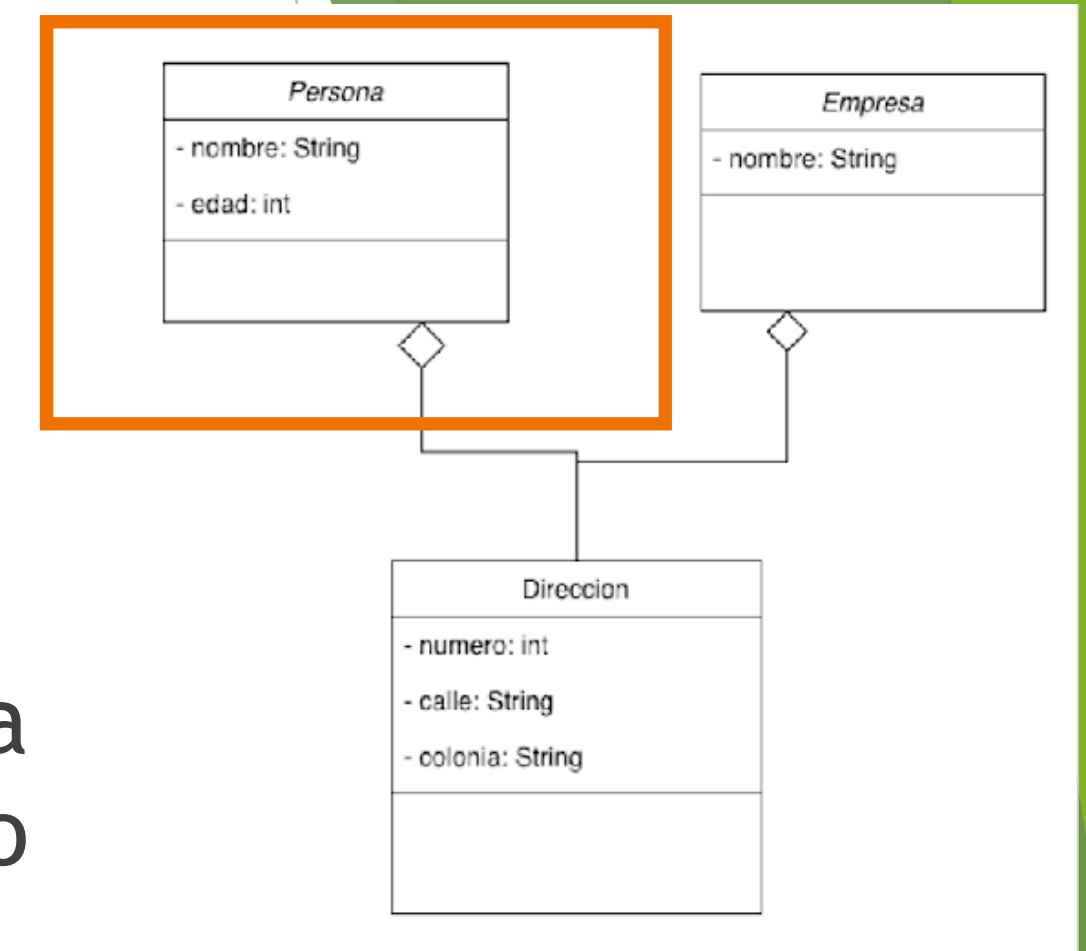
- ▶ La clase Dirección tiene los atributos calle, numero y colonia.
- ▶ La clase Dirección contiene el constructor, métodos gets y sets



Clase Persona

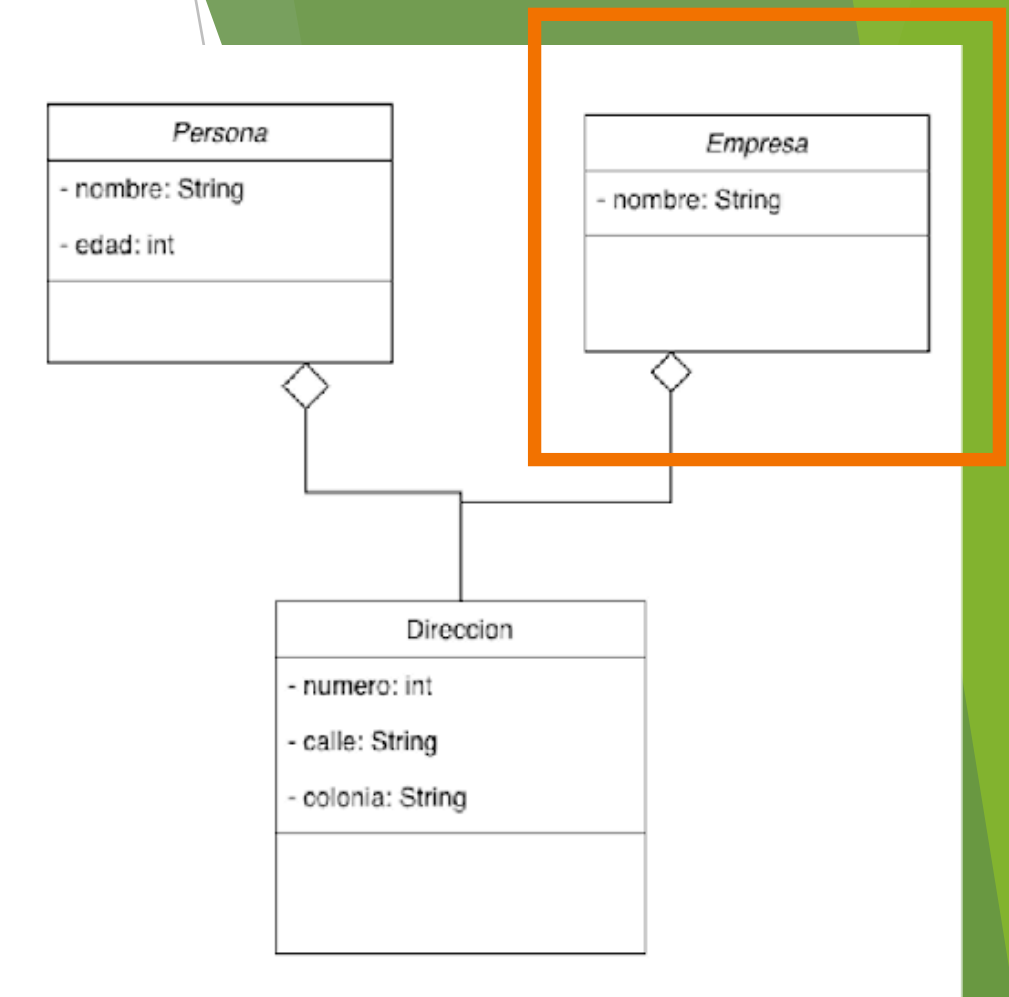
```
public class Persona {  
    private String nombre;  
    private int edad;  
    private Direccion direccion;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public Direccion getDireccion() {  
        return direccion;  
    }  
  
    public void setDireccion(Direccion direccion) {  
        this.direccion = direccion;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public int getEdad() {  
        return edad;  
    }  
    public void setEdad(int edad) {  
        this.edad = edad;  
    }  
}
```

- ▶ La clase Persona tiene una variable de instancia de tipo Dirección.
- ▶ Como se tiene una variable de tipo Dirección en la clase Persona, puede usar la referencia Dirección, que es agregación en este caso, para invocar la clase Dirección.
- ▶ Ahora se puede obtener todas las propiedades de ambas clases.



Clase Empresa

```
public class Empresa {  
    private String nombreEmpresa;  
    private Direccion direccion;  
  
    public Direccion getDireccion() {  
        return dirección;  
    }  
  
    public void setDireccion(Direccion direccion) {  
        this.dirección = direccion;  
    }  
  
    public String getNombreEmpresa() {  
        return nombreEmpresa;  
    }  
  
    public void setNombreEmpresa(String nombreEmpresa) {  
        this.nombreEmpresa = nombreEmpresa;  
    }  
}
```



- ▶ La clase Empresa tiene una variable de instancia de tipo Direccion. Como se tiene una variable de tipo Dirección en la clase Empresa, puede usar la referencia Dirección, que es agregación en este caso, para invocar la clase Direccion.
- ▶ Ahora se puede obtener todas las propiedades de ambas clases.

Clase Prueba

```
import java.util.*;
public class PruebaAgregacion {
    public static void main(String[] args) {
        String nombreP, calle, colonia, nE;
        int num, edad;
        //captura de datos para el objeto persona
        Scanner entrada = new Scanner(System.in);
        System.out.println("Introduce tu nombre: ");
        nombreP=entrada.nextLine();
        System.out.println("Introduce tu edad: ");
        edad=entrada.nextInt();
        entrada.nextLine();
        System.out.println("Introduce tu calle: ");
        calle=entrada.nextLine();
        System.out.println("Introduce el numero: ");
        num=entrada.nextInt();
        entrada.nextLine();
        System.out.println("Introduce tu colonia: ");
        colonia=entrada.nextLine();

        Direccion d1 = new Direccion(calle,num,colonia);
        Persona p = new Persona(nombreP,edad);
        p.setDireccion(d1);
    }
}
```

Salida del código

```
Los datos de la persona son:
Nombre: Maria Lopez
Edad: 19
Dirección: Camino Verde 12 El Florido

Los datos de la Empresa son:
Nombre: Rei
Dirección: Blvd. Diaz Ordaz 12345 Hipodromo
```

Clase Prueba

```
Direccion d1 = new Direccion(calle,num,colonia);
Persona p = new Persona(nombreP,edad);
p.setDireccion(d1);

//captura de datos para el objeto direccion
System.out.println("Introduce el nombre de la empresa: ");
nE=entrada.nextLine();
System.out.println("Introduce tu calle: ");
calle=entrada.nextLine();
System.out.println("Introduce el numero: ");
num=entrada.nextInt();
entrada.nextLine();
System.out.println("Introduce tu colonia: ");
colonia=entrada.nextLine();

Direccion d2 = new Direccion(calle, num,colonia);
Empresa e = new Empresa();
e.setNombreEmpresa("1A");
e.setDireccion(d2);

//imprimir los objetos
System.out.println("Los datos de la persona son:");
System.out.println("Nombre: "+p.getNombre());
System.out.println("Edad: "+p.getEdad());
System.out.println("Dirección: " +p.getDireccion().getCalle() + " "+p.getDireccion().getNumero() + "
"+p.getDireccion().getColonia());

//imprimir los objetos
System.out.println();
System.out.println("Los datos de la Empresa son:");
System.out.println("Nombre: "+e.getNombreEmpresa());
System.out.println("Dirección: " +e.getDireccion().getCalle() + " "+e.getDireccion().getNumero() + "
"+e.getDireccion().getColonia());
}
```

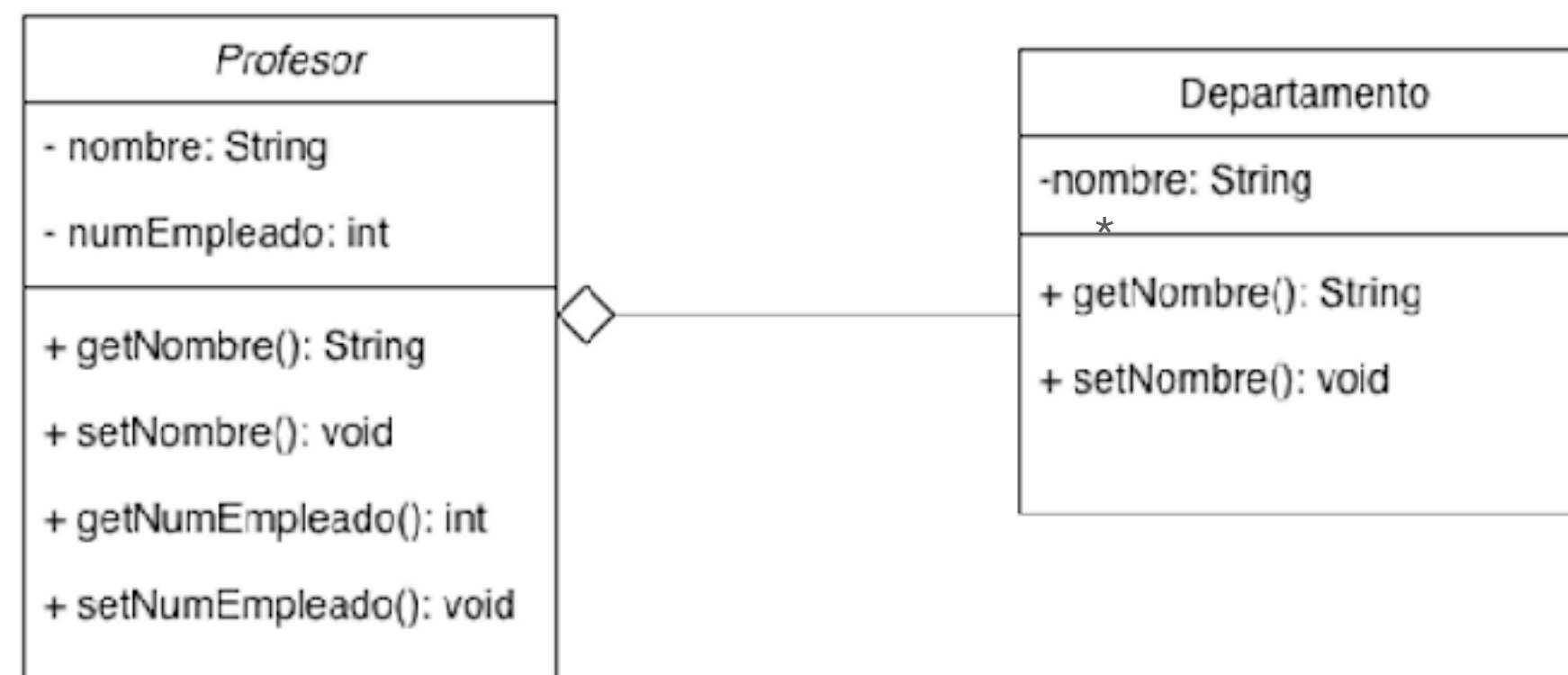
Salida del código

```
Los datos de la persona son:
Nombre: Maria Lopez
Edad: 19
Dirección: Camino Verde 12 El Florido

Los datos de la Empresa son:
Nombre: Rei
Dirección: Blvd. Diaz Ordaz 12345 Hipodromo
```

Agregación: Ejemplo (3)

- Agregar un ejemplo con multiplicidad

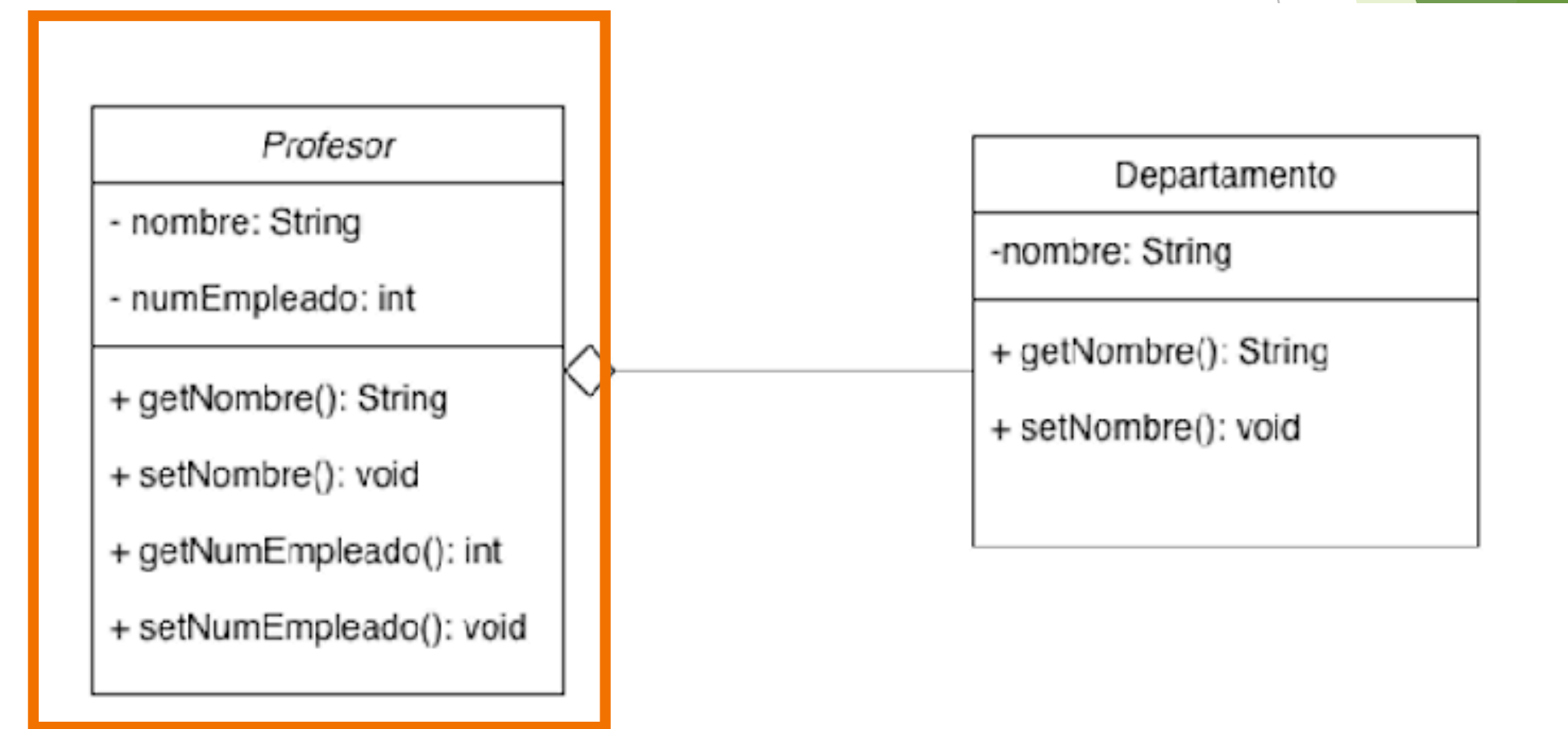


- Un profesor tiene un departamento
- Un departamento puede existir sin el profesor

Agregación: relación uno a uno

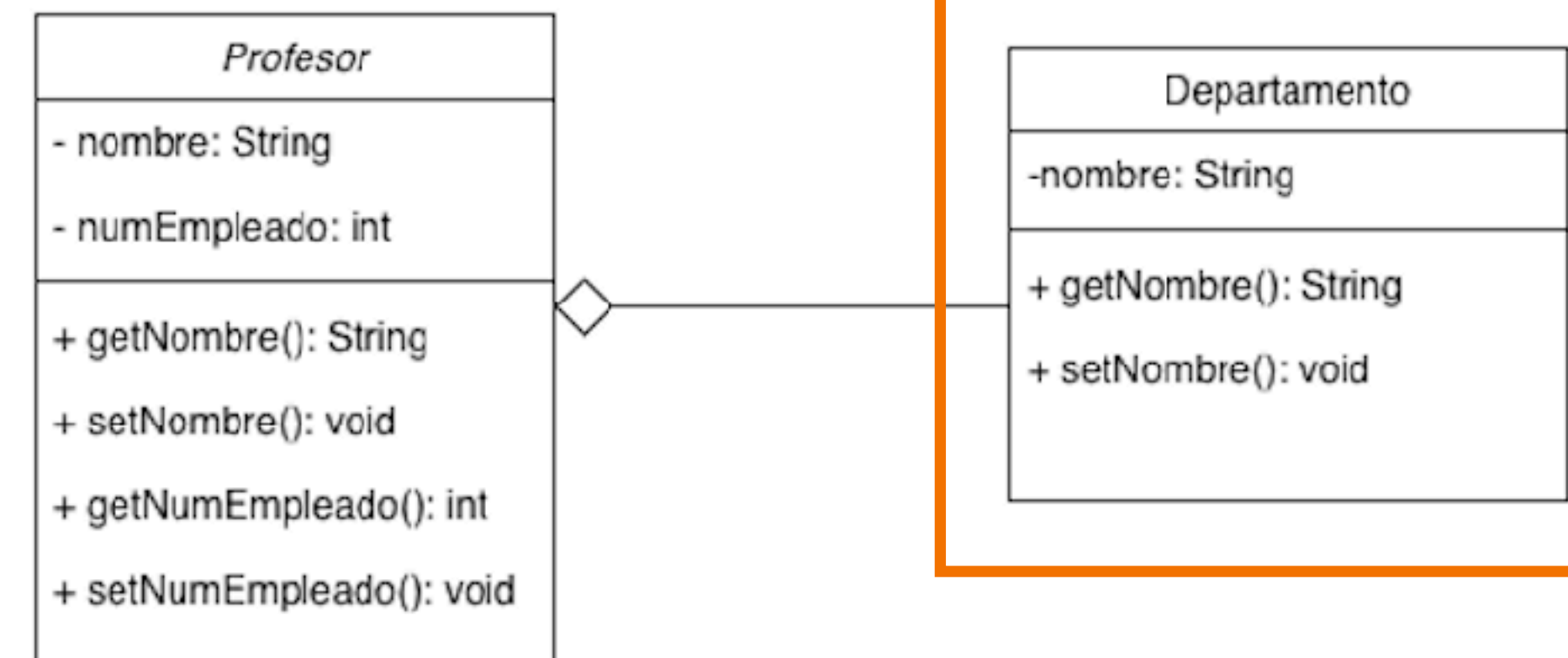
```
//agregación uno a uno
public class Profesor {
    private String nombre;
    private int numEmpleado;
    private Departamento departamento;

    public Profesor(String nombre, int numEmpleado, Departamento departamento) {
        this.nombre = nombre;
        this.numEmpleado = numEmpleado;
        this.departamento = departamento;
    }
    public String getNombre() {
        return nombre;
    }
    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
    public int getNumEmpleado() {
        return numEmpleado;
    }
    public void setNumEmpleado(int numEmpleado) {
        this.numEmpleado = numEmpleado;
    }
    public Departamento getDepartamento() {
        return departamento;
    }
    public void setDepartamento(Departamento departamento) {
        this.departamento = departamento;
    }
}
```

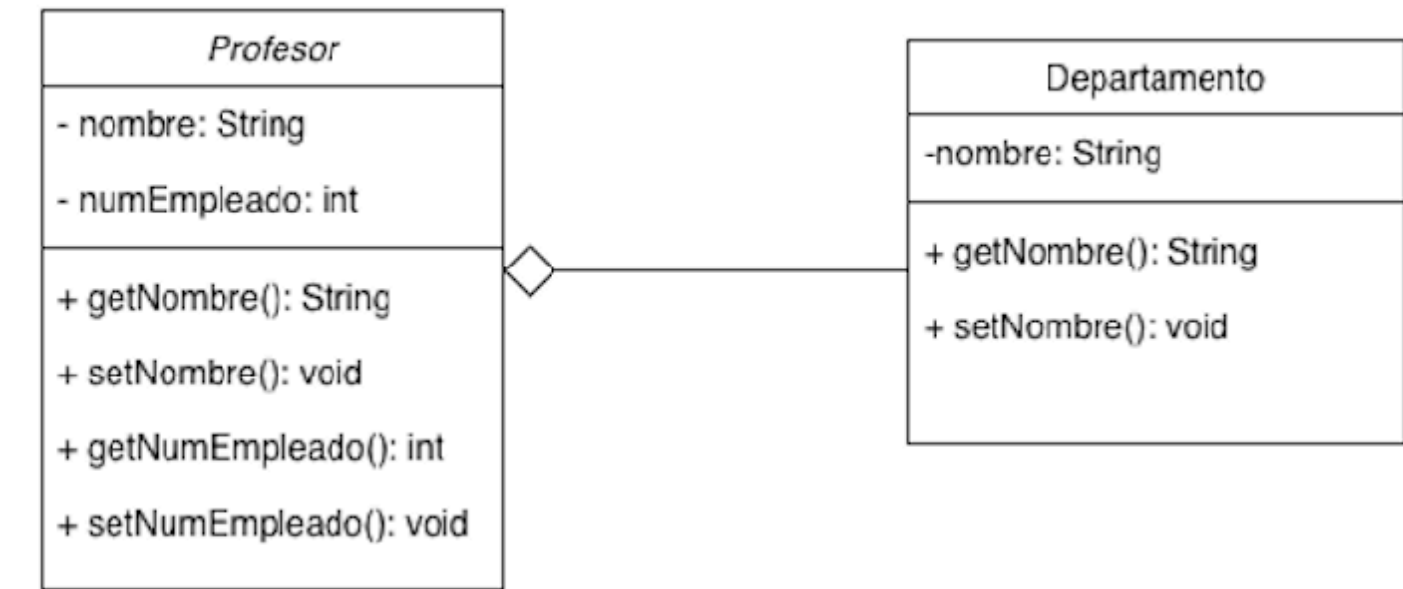


Agregación: relación uno a uno

```
public class Departamento {  
    private String nombre;  
    private Profesor jefeDepartamento;  
  
    public Departamento(String nombre) {  
        this.nombre = nombre;  
    }  
    public String getNombre() {  
        return nombre;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public Profesor getJefeDepartamento() {  
        return jefeDepartamento;  
    }  
    public void setJefeDepartamento(Profesor jefeDepartamento) {  
        this.jefeDepartamento = jefeDepartamento;  
    }  
}
```



Relación uno a uno



```
public class PruebaRelacionUnoAUno {
    public static void main(String[] args) {

        Departamento departamento = new Departamento("Matemáticas");
        Profesor profesor = new Profesor("Juan", 24342, departamento);
        profesor.setDepartamento(departamento);
        departamento.setJefeDepartamento(profesor);

        System.out.println("El jefe del departamento de " + departamento.getNombre() + " es: " +
departamento.getJefeDepartamento().getNombre());
    }
}
```

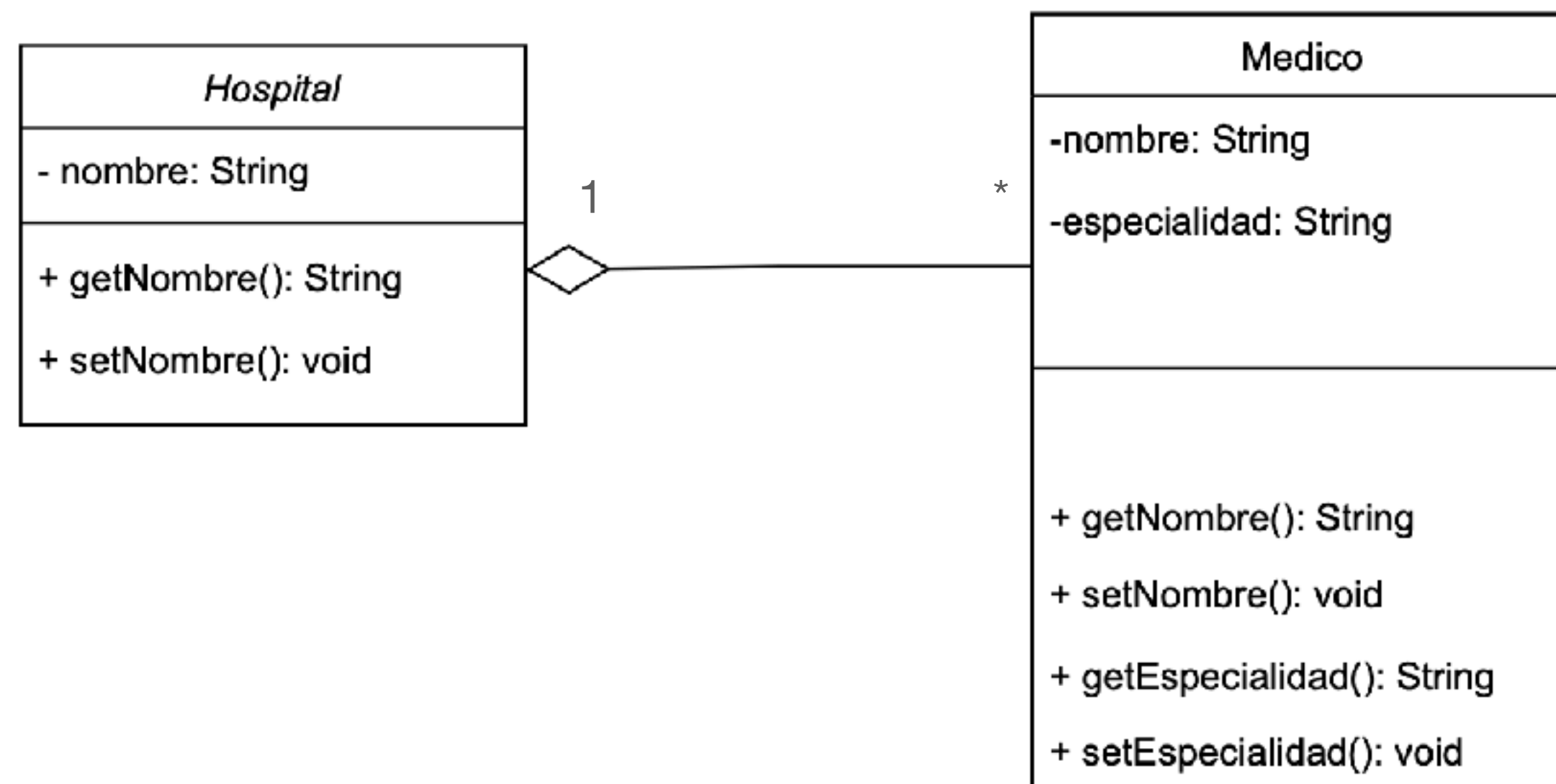
Salida del código

```
El jefe del departamento de Matemáticas es: Juan
```

- ▶ Entonces un objeto podrá acceder a los datos de otro objeto.
- ▶ Por ejemplo, una entidad profesor modelada como un objeto podría acceder/conocer los nombres de todos los departamentos en los que trabaja.
- ▶ Y un objeto de departamento puede acceder/conocer los nombres/detalles de todos los profesores que trabajan en él. La funcionalidad/servicios de un objeto puede ser accesible para otro objeto.

Agregación: Ejemplo (3)

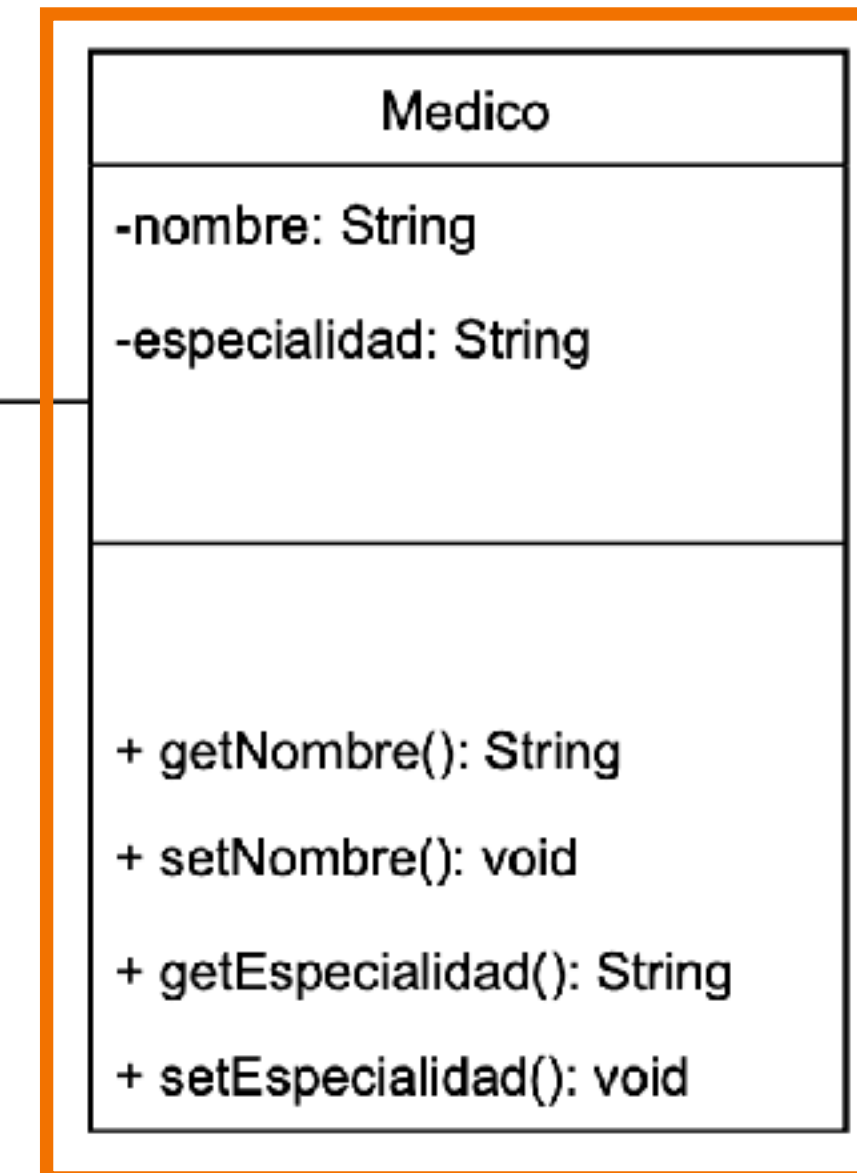
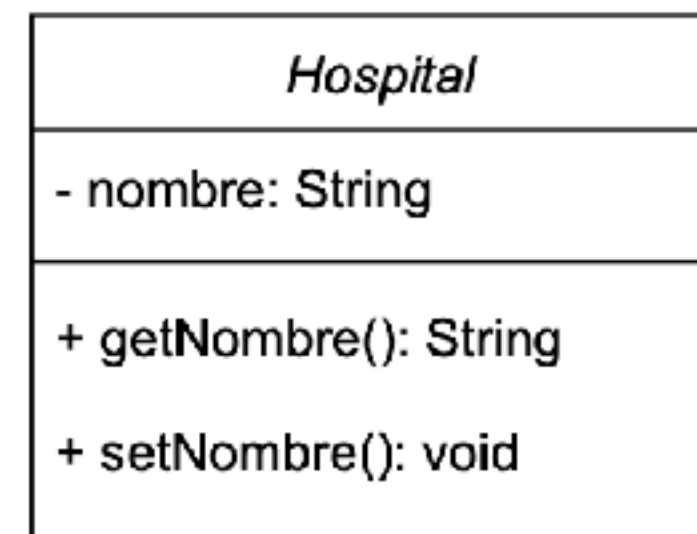
- Agregar un ejemplo con multiplicidad



- Un hospital tiene muchos médicos. Un médico puede existir sin el hospital

Agregación: Ejemplo (2)

```
public class Medico {  
    private String nombre;  
    private String especialidad;  
  
    public Medico(String nombre, String especialidad) {  
        this.nombre = nombre;  
        this.especialidad = especialidad;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public String getEspecialidad() {  
        return especialidad;  
    }  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
    public void setEspecialidad(String especialidad) {  
        this.especialidad = especialidad;  
    }  
}
```



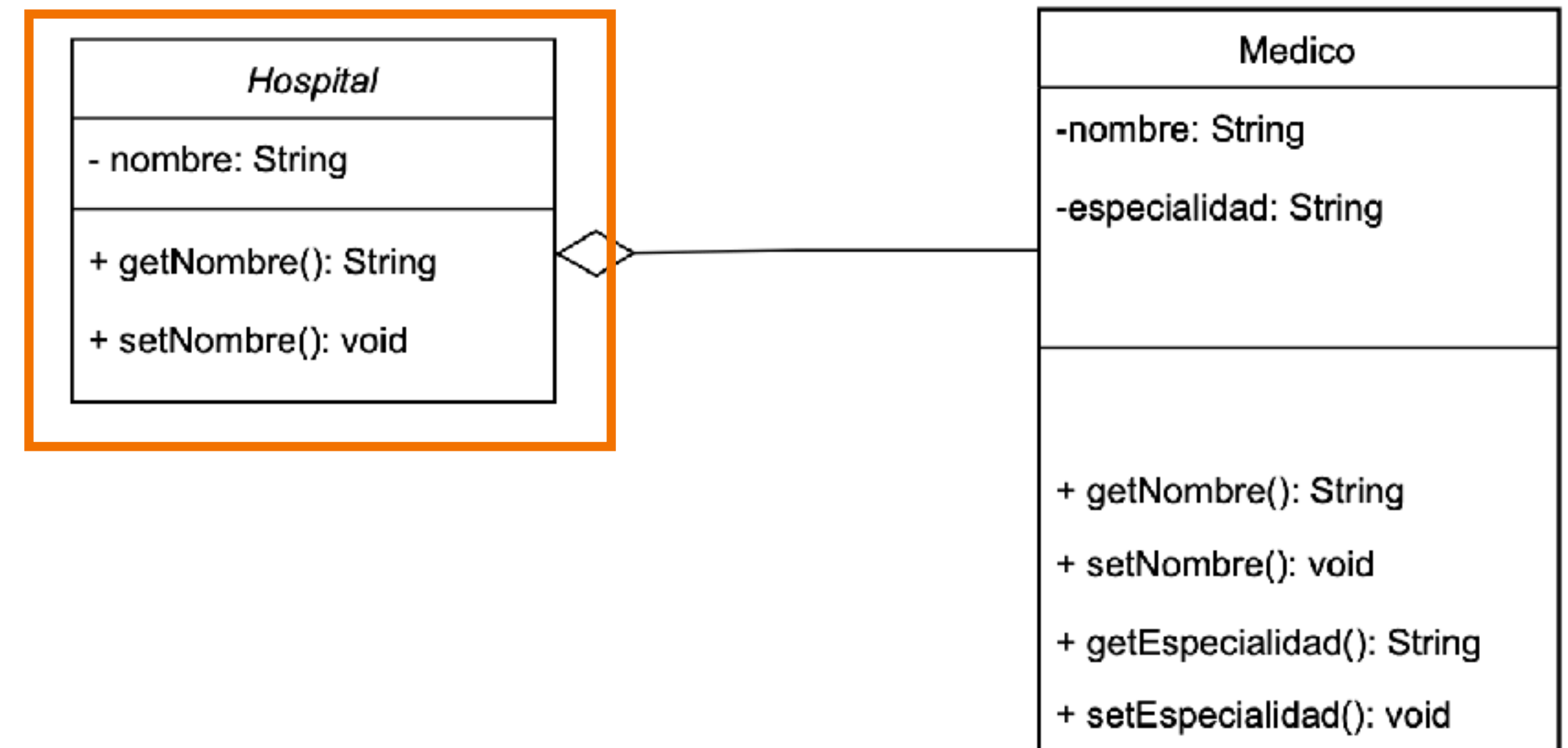
Agregación: Ejemplo (2)

```
public class Hospital {
    private String nombre;
    private Medico[] medicos;
    private int contador;
    private int capacidad;

    public Hospital(String nombre, int capacidad) {
        this.nombre = nombre;
        this.capacidad = capacidad;
        this.medicos = new Medico[capacidad];
        this.contador = 0;
    }

    public void agregarMedico(Medico medico) {
        if (contador < capacidad) {
            medicos[contador] = medico;
            contador++;
        }
        else
        {
            System.out.println("Excedio la cantidad de médicos");
        }
    }

    public String getNombre() {
        return nombre;
    }
    public Medico[] getMedicos() {
        return medicos;
    }
    public int getContador(){
        return contador;
    }
}
```



Agregación: Ejemplo (2)

```
public class PruebaAgregacionMultiplicidad {  
    public static void main(String[] args) {  
        // Crear un hospital  
        Hospital hospital = new Hospital("Hospital Central",2);  
  
        // Crear algunos médicos  
        Medico medico1 = new Medico("Dr. Pérez", "Pediatria");  
        Medico medico2 = new Medico("Dr. Gómez", "Cardiología");  
  
        // Agregar los médicos al hospital  
        hospital.agregarMedico(medico1);  
        hospital.agregarMedico(medico2);  
  
        // Mostrar información del hospital y sus médicos  
        System.out.println("Hospital: " + hospital.getNombre());  
        System.out.println("Médicos:");  
  
        for (int i = 0; i < hospital.getContador(); i++) {  
            Medico medico = hospital.getMedicos()[i];  
            System.out.println("- " + medico.getNombre() + " (" + medico.getEspecialidad() + ")");  
        }  
    }  
}
```

Salida del código

```
Hospital: Hospital Central  
Médicos:  
- Dr. Pérez (Pediatria)  
- Dr. Gómez (Cardiología)
```